

微机系统与接口——

第八章 80386微处理器

东南大学信息科学与工程学院



王东明

本章目标 • OBJECTIVE



- 熟悉386内部结构
- 掌握80386的保护模式（重点）
- 掌握80386的存储器管理（重点）
- 掌握80386的段页式寻址（重点）

学习收获 • GAINS

- 学习操作系统、编译原理的基础；
- 学习32位汇编的基础。



学习难点 • Difficulties

- 保护模式中虚拟存储器的概念。
- 存储器的段页式寻址：逻辑地址转换为物理地址的过程。
- 386的安全机制。



学习方法 • METHOD

- 深刻理解虚拟存储器的概念，理解386的解决8086存在问题的思路。
- 掌握多级链表查询：根据序号找到表项，根据表项的格式，分析其属性和其对应的表的基地址。



前言

1

80386的存储器管理与8086有什么不同？

2

80386的内部寄存器与8086有哪些不同？

3

详细说明80386的逻辑地址转换成物理地址的过程。





1

80386及其特点

2

80386工作模式

3

80386内部寄存器

4

80386存储器管理

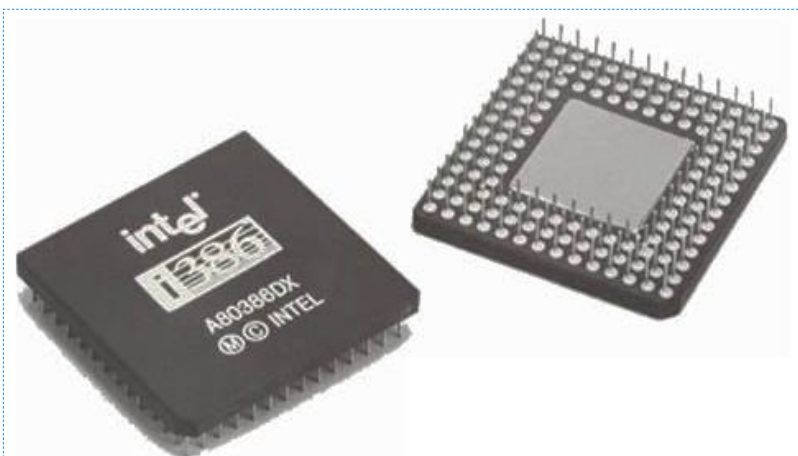
80386及其特点

◆ 80386的概述

◆ 80386内部结构

◆ 80386的特点

80386

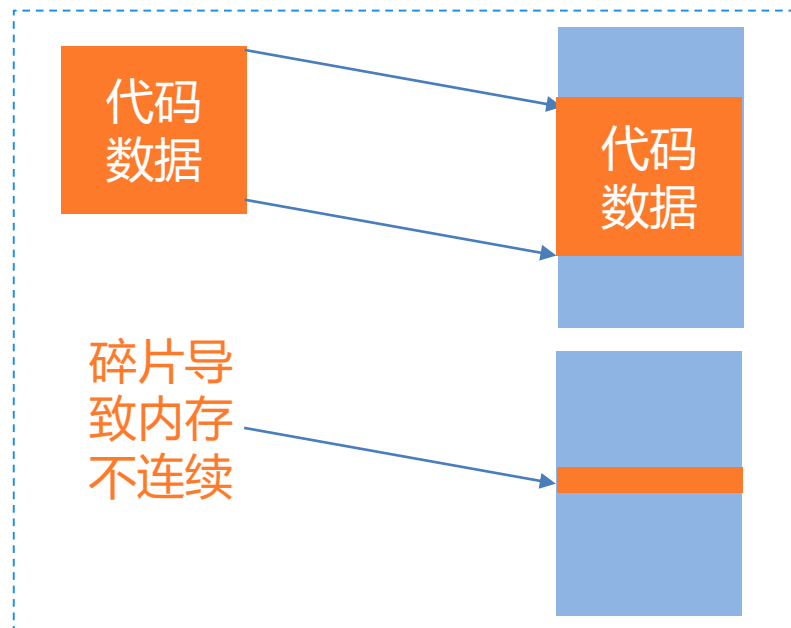


- 1985年10月，Intel推出了**第一个高性能全32位的CPU——80386**。
- 体系结构的突破：从**16位体系结构过渡到32位体系结构**。

至今，CPU的体系结构仍没有革命性突破。

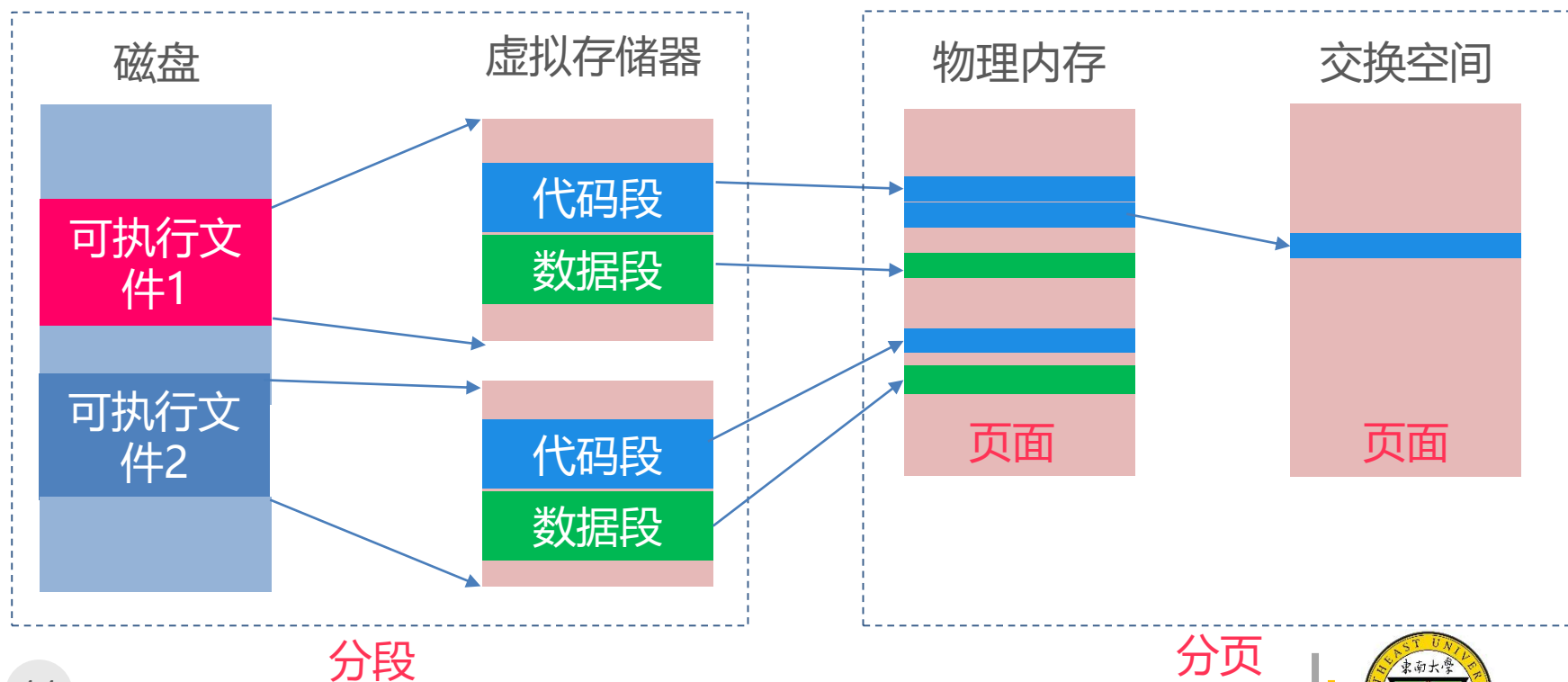
16位系统架构的缺点

- **单任务：** 16位CPU只支持单任务。
- **内存小：** 运行时程序全部加载到内存，当程序大于内存时无法运行。
- **安全差：** 对内存访问没有太多的限制，应用程序容易修改操作系统等内存空间，导致系统崩溃。
- **效率低：** 碎片化问题严重。



80386的突破

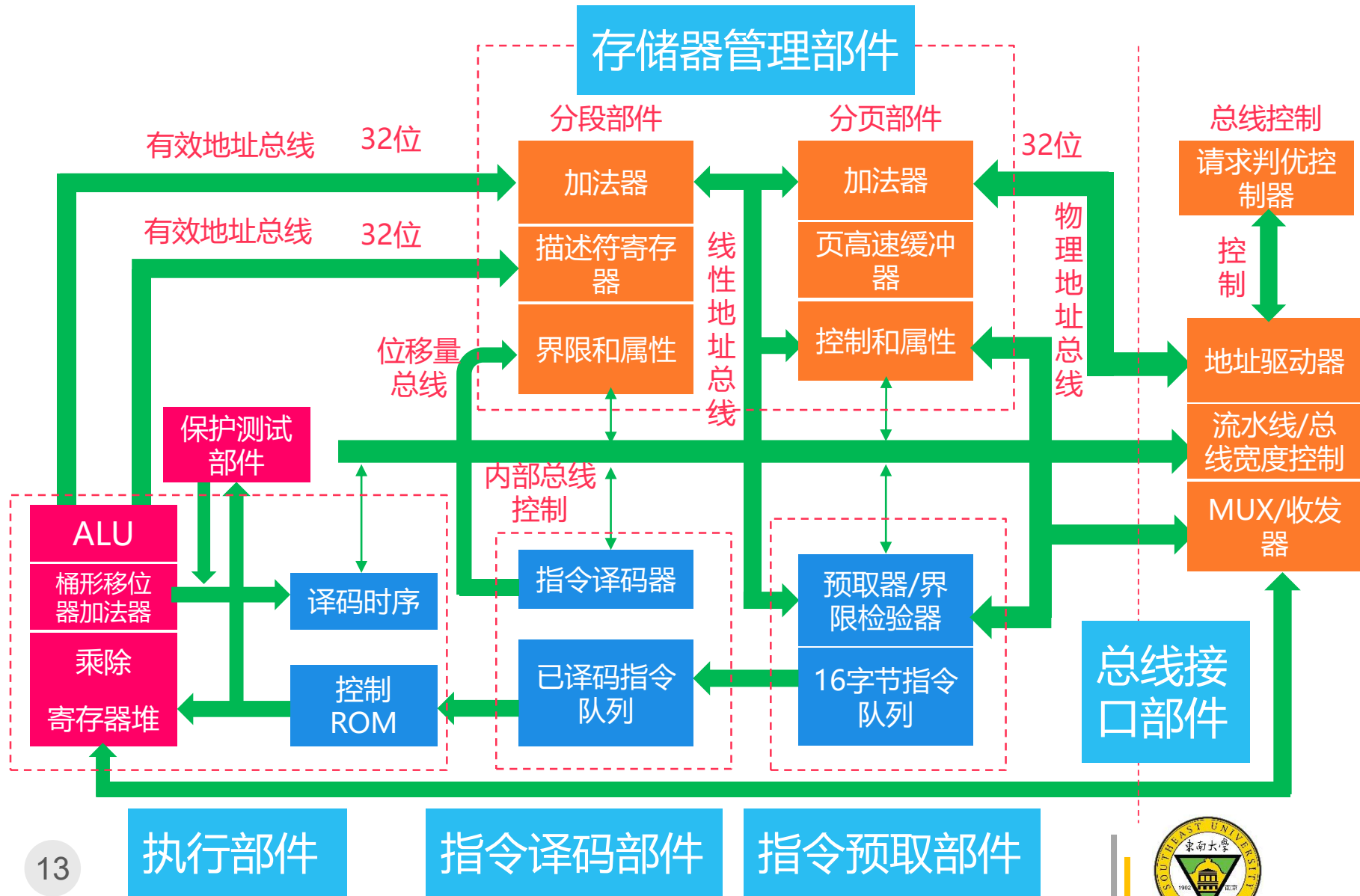
- **虚拟存储器**：逻辑内存与物理内存分离，虚拟存储器远大于物理内存。
- **安全**：地址空间隔离，也允许共享内存。
- **多任务**：允许更高效的进程创建，程序执行时只需部分在内存。



80386及其特点

- ◆ 80386的概述
- ◆ 80386内部结构
- ◆ 80386的特点

80386内部结构



80386及其特点

- ◆ 80386的概述
- ◆ 80386内部结构
- ◆ 80386的特点

80386的特点

工作方式

- 80386可以工作在三种方式：**实地址方式、保护方式，虚拟8086方式。**
- 硬件支持**段式管理和页式管理**，易于实现虚拟存储系统。每个任务可运行多达16K个段，每个段最大可为4G字节，故每个任务最大可达**64MM字节**。



80386的特点

特权级

- 4级特权级：0级最优先，其次为1，2和3级。0、1和2级用于操作系统程序，3级用于用户程序。
- 完善的特权检查机制，既能实现资源共享又能保障程序和数据的安全和保密以及任务间的隔离。
- 硬件支持多任务，一条指令可以完成任务转换，转换时间在 $17\mu\text{s}$ 以内。





1

80386及其特点

2

80386工作模式

3

80386内部寄存器

4

80386存储器管理

80386工作模式

实模式

- 8086工作方式基本相同

保护模式

- 实现虚拟存储
- 支持多用户、多任务

虚拟8086模式

- 多任务运行DOS
- 与8086兼容又不同于实地址



实模式

1 实模式是80386最基本的工作方式，与8086工作方式基本相同。

2 加电后进入实模式，为保护模式所需数据结构做配置和准备。

特点

- 1MB存储空间，段加偏移访问存储器，段最大64KB；
- 不采用分页部件，即不支持虚拟存储器；
- 单任务，代码段和数据段没有保护机制。



80386保护模式

- ◆ 保护模式下的分段
- ◆ 保护模式下的分页
- ◆ 80386的安全机制

保护模式——虚拟地址空间的分段



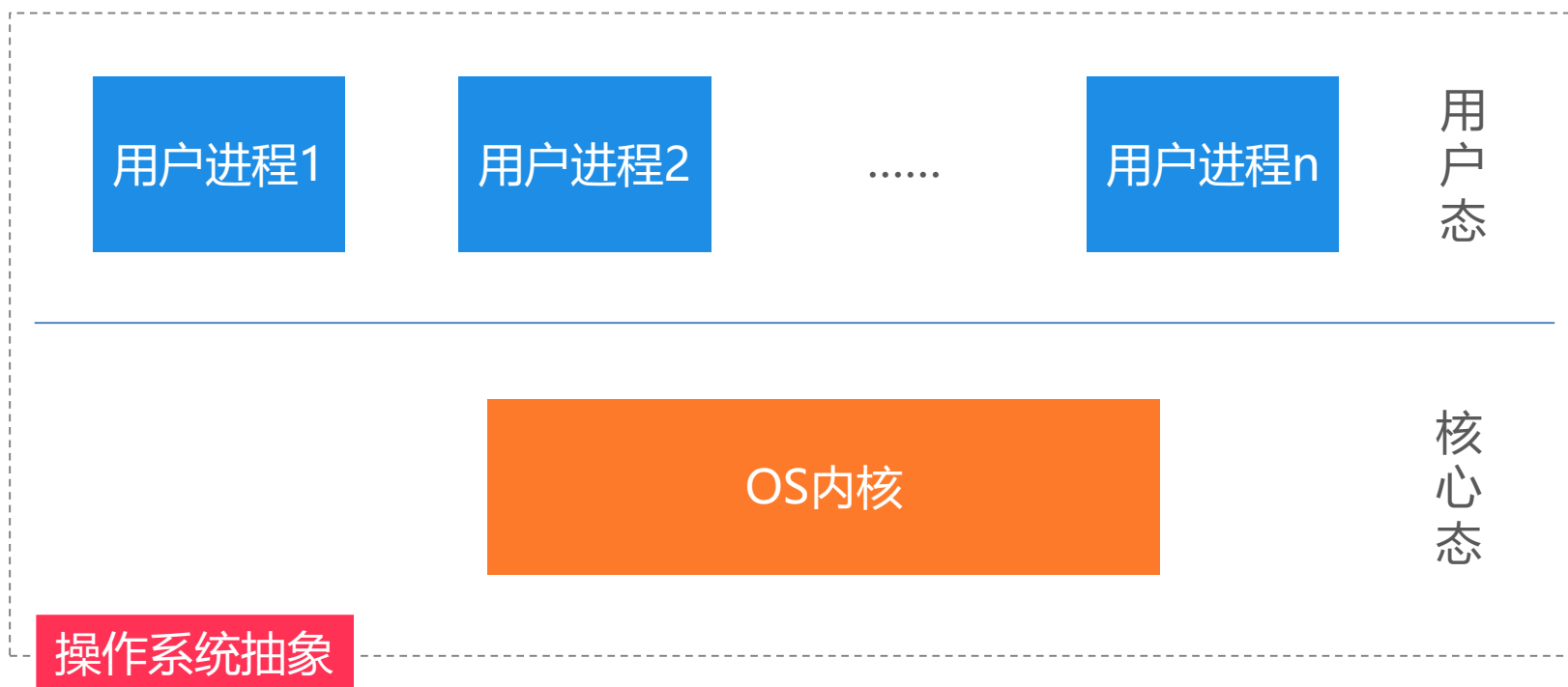
为什么要分段？

386分段与8086有何不同？

- 解决多任务的隔离问题；
- 解决程序的编译问题。

保护模式：进程与任务

进程即任务，是对CPU的一个抽象。



保护模式：虚拟存储器的概念

地址空间是对物理内存的抽象，每个进程有自己独立的地址空间。

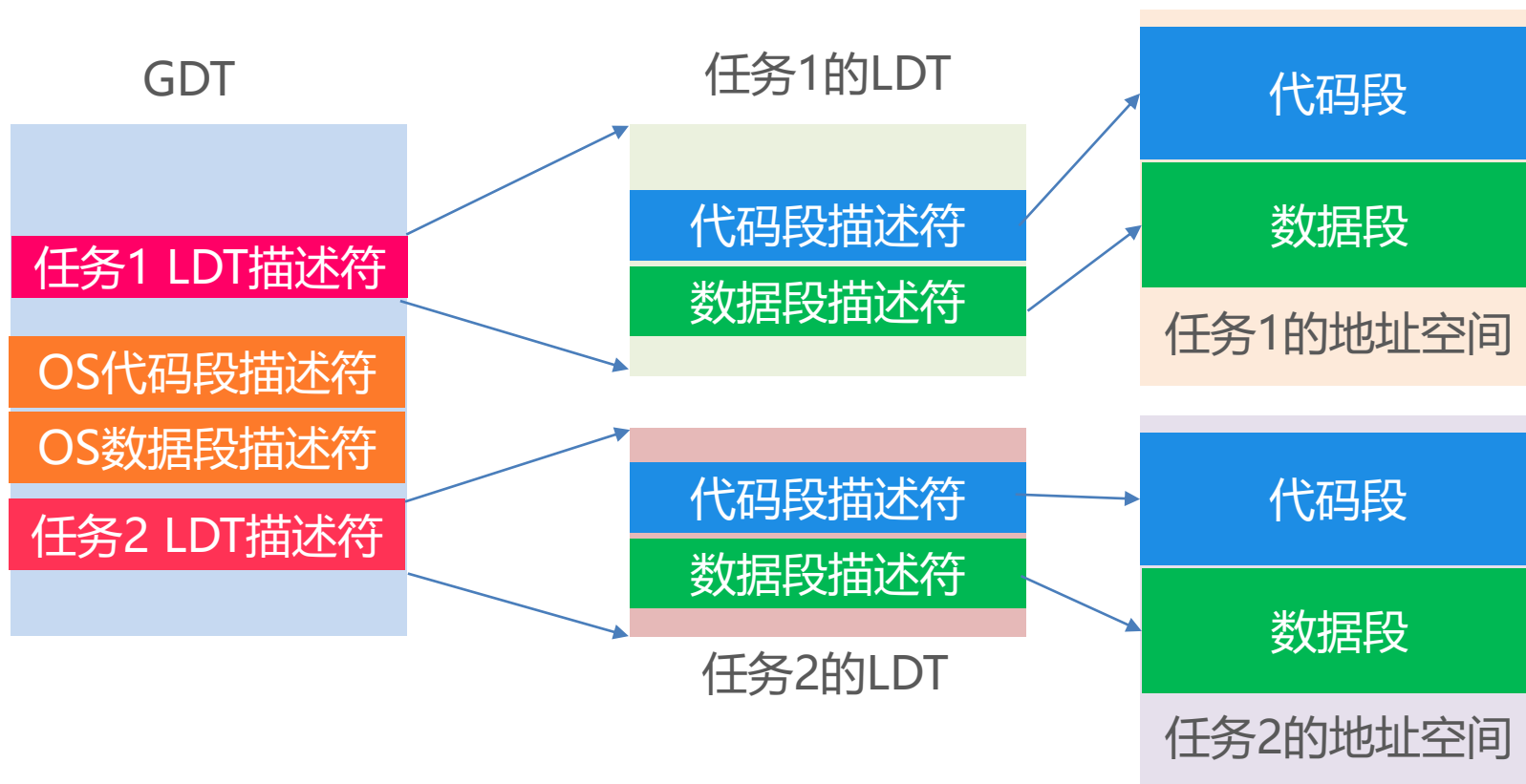
特点

- 地址线性排列，又称为线性地址空间；**存储器分段是必须的**，没有分页的情况下，线性地址即物理地址；
- 现有的操作系统均采用分段、分页。有分页的情况下，线性地址需要经过映射，转换为物理地址。因此，线性地址空间也被称为虚拟存储空间。



保护模式：GDT与LDT

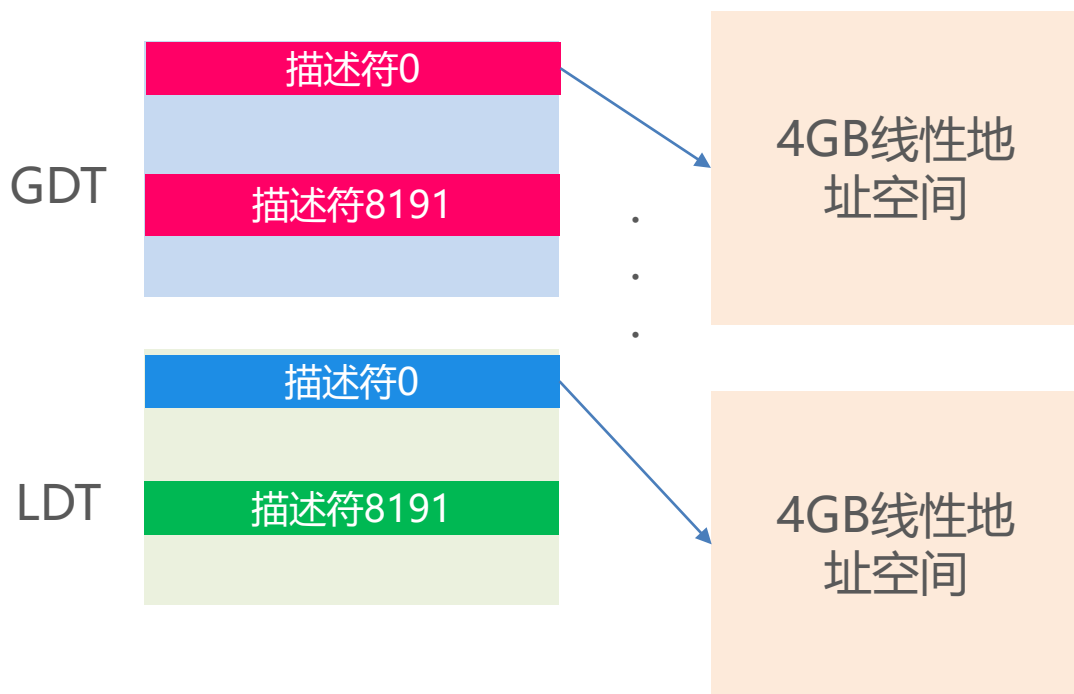
全局描述符表：GDT，唯一。局部描述符表：LDT，每个任务有。



保护模式下的虚拟存储器大小

GDT和LDT中最多各有 2^{13} 个描述符，每个描述符可寻址4GB。

80386理论上可以寻址： 2^{46} B的虚拟存储器。



保护模式：分段的优点

- 1 每个任务有自己的地址空间，任务隔离，各自独立。
- 2 易于保护数据不被未经授权的用户访问，程序不被别的进程破坏。
- 3 LDT和GDT实现进程的虚拟地址的隔离。



80386保护模式

- ◆ 保护模式下的分段
- ◆ 保护模式下的分页
- ◆ 80386的安全机制

分页——从线性地址到物理地址映射

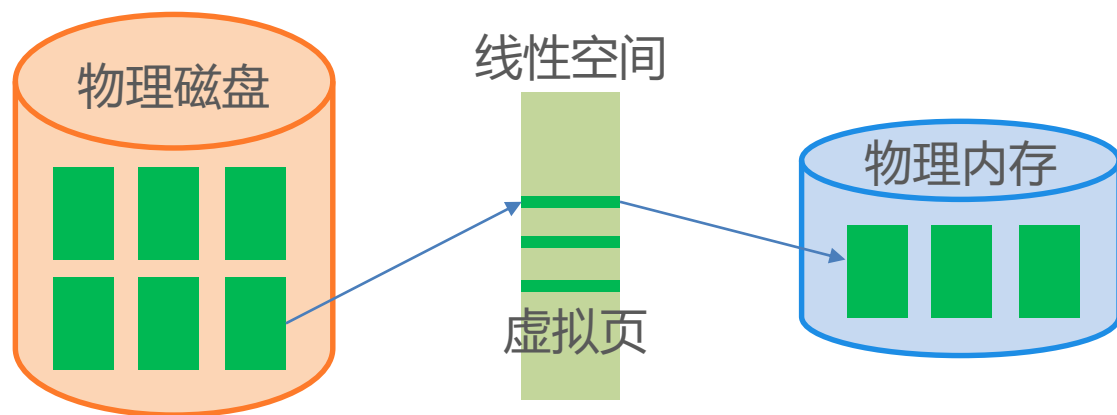


为什么要分页？

- 提高内存的使用效率；
- 解决小内存运行大程序。

保护模式下的分页

- 1 磁盘分扇区，虚拟存储空间被划分成页，os把物理内存分为页。
- 2 这些分页只是os为了管理方便，在逻辑上划分的一种数据结构。
- 3 虚拟内存的页可以和物理内存的页对应起来，产生映射关系。



页为什么是4KB?

- 1 对于磁盘，扇区小，开销大，传送性能低。
- 2 对于内存，页小，开销大；页大，会导致碎片，申请内存多用到的少。
- 3 当前磁盘扇区4KB，Windows，Linux内存页默认4KB。



拷贝1024个1KB的文件快还是1MB文件快?

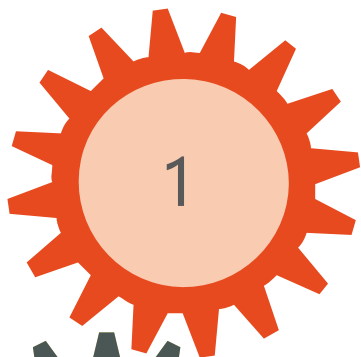


虚拟页

虚拟页到底是实际存在的还是虚拟的?



虚拟页



程序运行前，它的数据和代码是在可执行文件中，运行时需要把可执行文件加载到内存。



硬盘上的文件逻辑上划分为4K的页（实际由加载器加载到内存），这就是虚拟页里面实际的东西。



但是，程序在运行是可能会申请内存，这时需新的虚拟页来映射。所以，虚拟页应有3种状态。

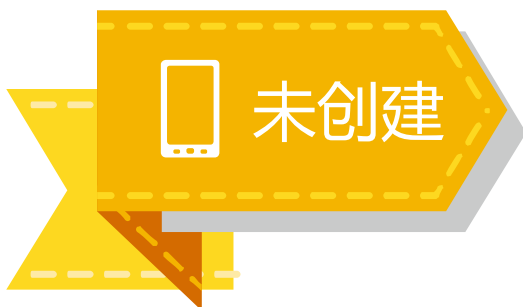
虚拟页的3种状态



虚拟页面被创建并被加载到物理内存，和物理页之间存在映射关系。

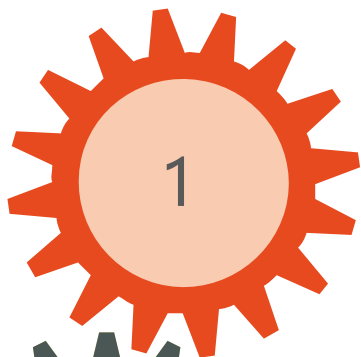


虚拟页被创建，但未被加载到内存或已被调出内存，和物理页面之间没有映射关系。



虚拟页面未创建，未被访问所以未加载或是调用malloc分配内存，仅运行时才被创建。

存储器映射



加载应用程序到内存时，因为和虚拟地址有关，我们需要把应用程序文件和虚拟内存关联起来。

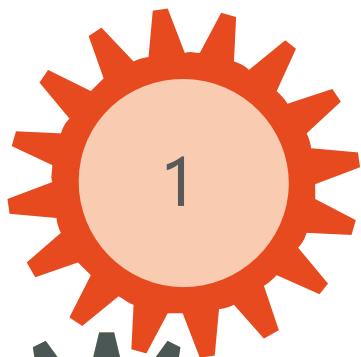


在Linux中称为存储器映射，同时提供了一个mmap的系统调用来实现此功能。



文件被分成页面大小，每页含一个虚拟页内容。页面按需调度，CPU发出访问请求时，创建虚拟页并加载到内存。

存储器映射



启动一个进程时，加载器完成文件映射的功能，所以执行文件又可称为映像文件。



实际上加载器并不真正负责从磁盘加载数据到内存，因为和虚拟内存建立了映射关系，操作是虚拟内存自动进行的。

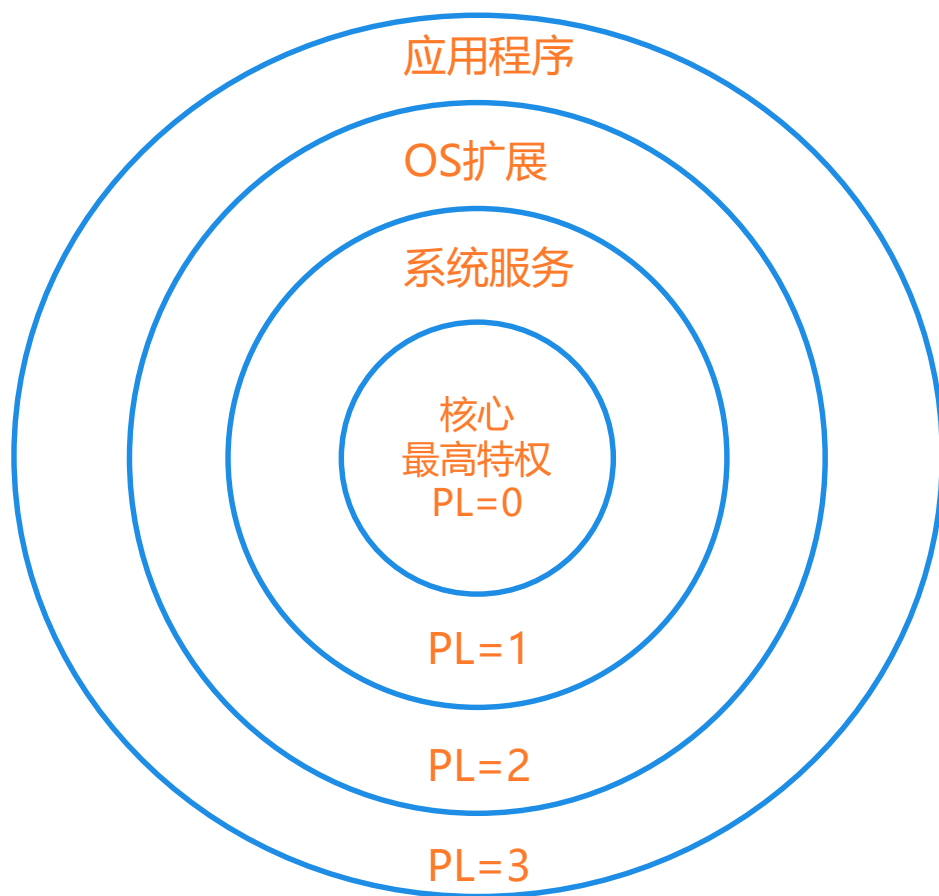


正是有了存储器映射的存在，使得我们可以很方便的将程序和数据加载到内存中。

80386保护模式

- ◆ 保护模式下的分段
- ◆ 保护模式下的分页
- ◆ 80386的特权机制

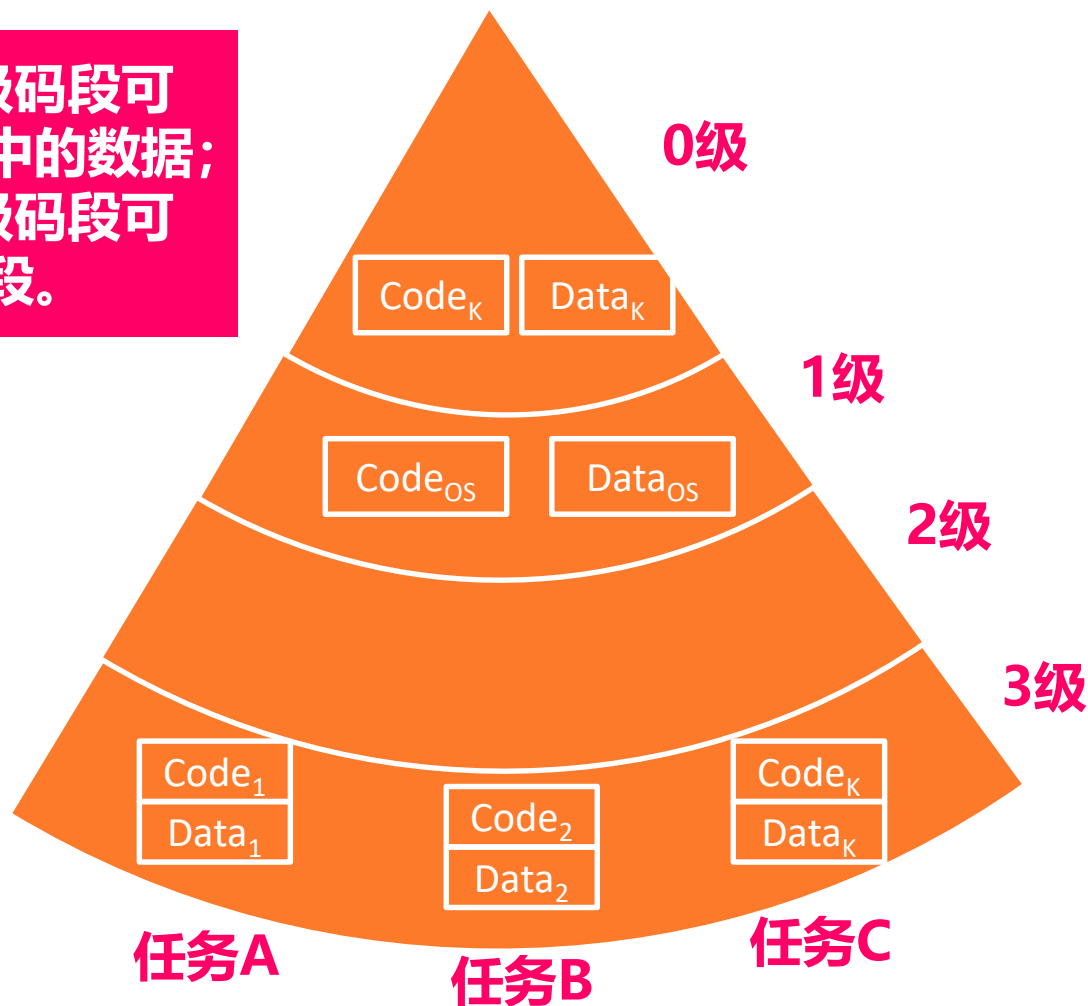
80386的特权机制



80386为系统设置了4个特权等级 (privilege level)，其取值由0到3，0为最高特权级。

保护模式下的特权规则

规则1：高特权级码段可访问低特权级段中的数据；
规则2：低特权级码段可调用高特权级码段。



保护模式：小结

分段解决进程的虚拟地址空间的隔离，使程序员从复杂纷繁的存储器管理解脱，只需要关注程序设计，不关心内存不够用。

分页是解决物理内存的使用效率，与程序的执行密切相关，与程序员设计程序无关，程序员不需要关心程序是如何被执行的，从而实现了真正的虚拟存储器。

分段，分页



检测点



1 80386CPU的地址线有_____位，可以寻址_____的物理存储器。

2 80386的工作模式包括_____、_____、_____。

3 保护模式下80386的可以寻址的虚拟存储器最大为_____。





1

80386及其特点

2

80386工作模式

3

80386内部寄存器

4

80386存储器管理

8 个32位通用寄存器

EAX		AH AL
EBX		BH BL
ECX		CH CL
EDX		DH DL
ESI		SI
EDI		DI
EBP		BP
ESP		SP



6个16位段寄存器

16位的段寄存器

段描述符高速缓存（64位，自动加载）

CS	段选择子	段基地址	段界限和属性
SS	段选择子	段基地址	段界限和属性
DS	段选择子	段基地址	段界限和属性
ES	段选择子	段基地址	段界限和属性
FS	段选择子	段基地址	段界限和属性
GS	段选择子	段基地址	段界限和属性



段寄存器不再直接存放段地址，而是一个选择子，通过它间接查找到段描述符，进而得到段基地址。

指令指针和标志寄存器

32位的指令指针（EIP）和标志寄存器（EFLAGS）各一个。

实模式时采用16位的指令指针IP（EIP低16位）。

EFLAGS的低12位与8086标志寄存器FLAGS完全相同。



系统地址寄存器和系统段寄存器

GDTR 48位

32位线性基地址

16位界限值

IDTR 48位

32位线性基地址

16位界限值

LDTR

16位选择子

段基地址

段界限和属性

TR

16位选择子

段基地址

段界限和属性

GDTR和IDTR在系统唯一，每任务一个LDTR和TR（Intel建议）。



在多CPU系统中，GDTR的个数等于CPU的个数。



控制寄存器

32位的控制寄存器CR0、CR2和CR3，保存各种全局性状态。

CR0是最常用的一个控制寄存器，选择工作模式。

CR3为页目录基址寄存器，存放页目录表的物理基地址。

PG

.....

ET

TS

EM

MP

PE

PG	PE	80386的工作模式选择
0	0	实模式
0	1	分段，不分页的保护模式
1	1	分段分页的保护模式





1

80386及其特点

2

80386工作模式

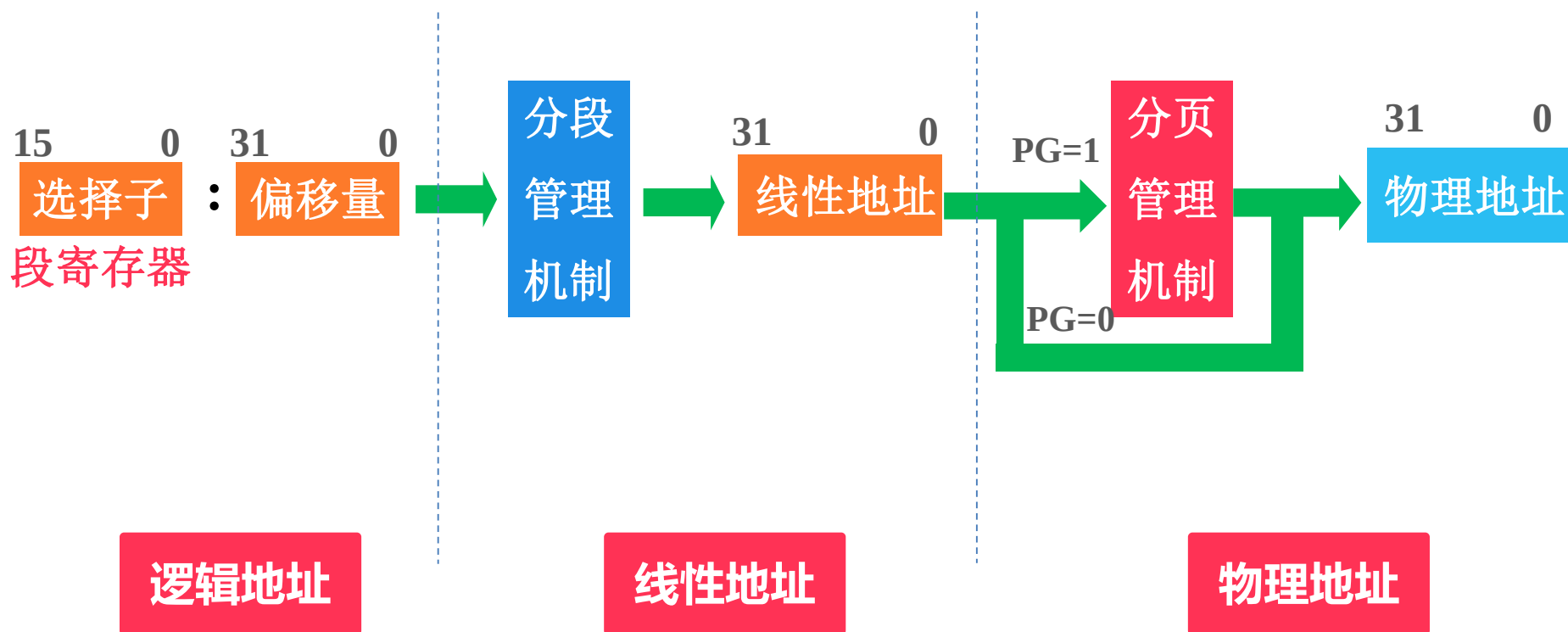
3

80386内部寄存器

4

80386存储器管理

80386的存储管理机制



80386存储器管理

◆ 分段

◆ 分页

◆ 举例

80386的分段管理机制

逻辑
地址

段寄存器

偏移地址



线性
地址

段基地址



偏移地址

段基地址由段寄存器间接获取，偏移地址由指令中的寻址方式获取。

描述一个段需要的数据结构

仅段寄存器来确定段基地址还不够，还需描述段长度、段属性如访问权等。



描述符、描述符表和选择子

描述符

描述段属性的8个字节的存储单元。

描述符表

存放描述符的特殊段。最大64KB，最多存放8192个描述符。

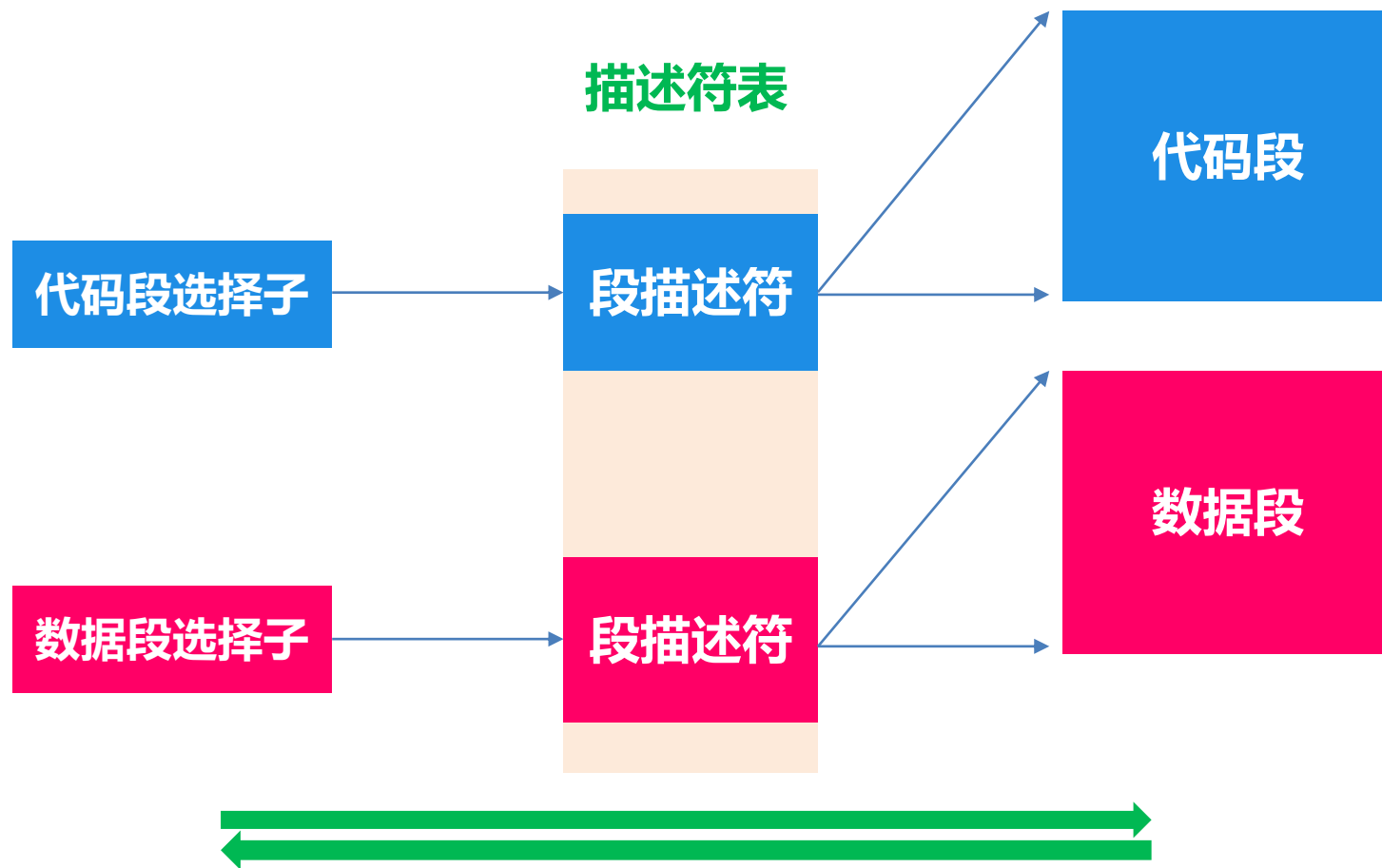
段选择子

16位段寄存器，其高13位是描述符在描述符表中的索引号。

描述符表分为全局描述符表GDT、局部描述符表LDT和中断描述符表IDT。



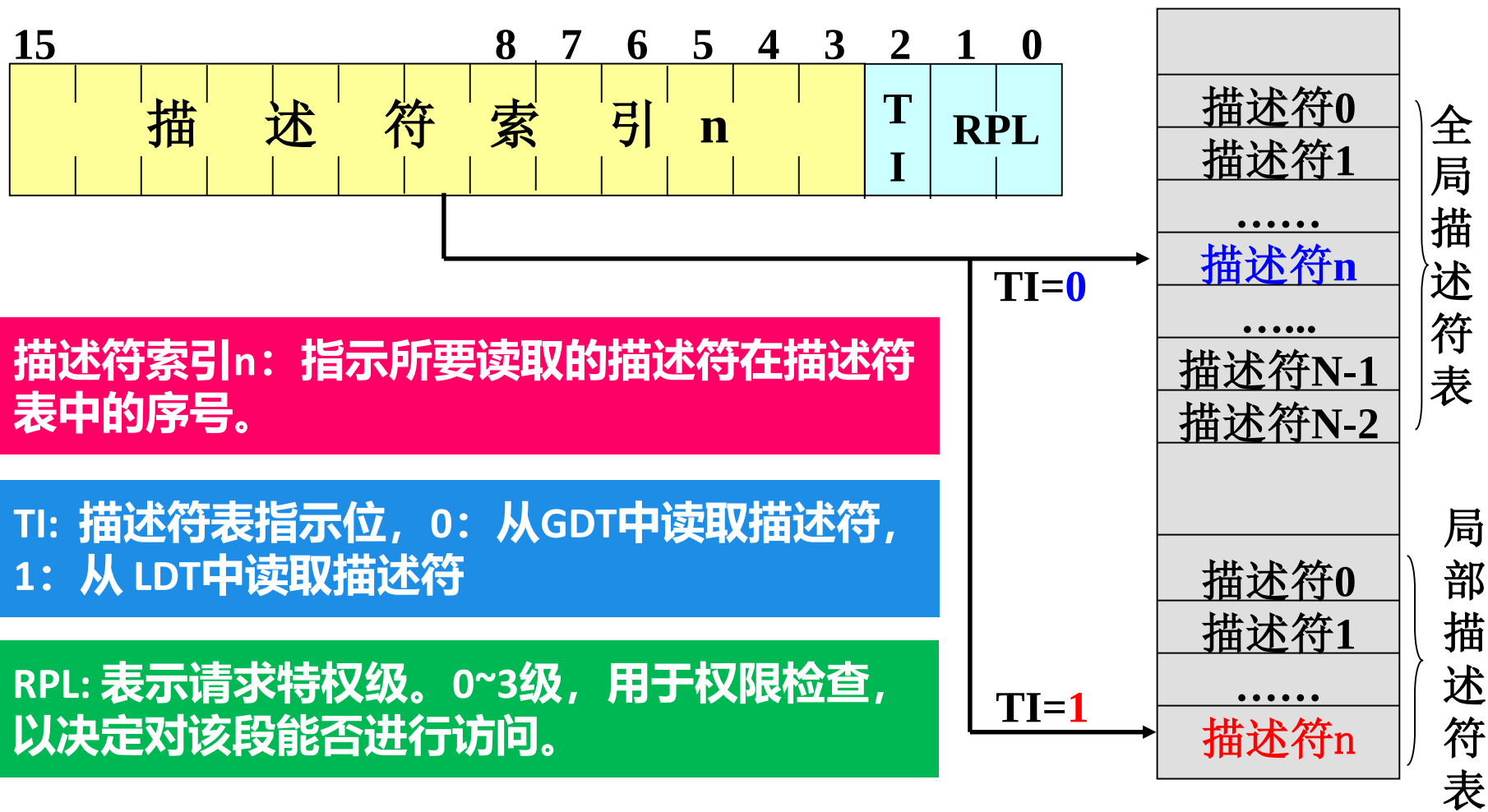
保护方式下段基地址的产生



描述符表



选择子

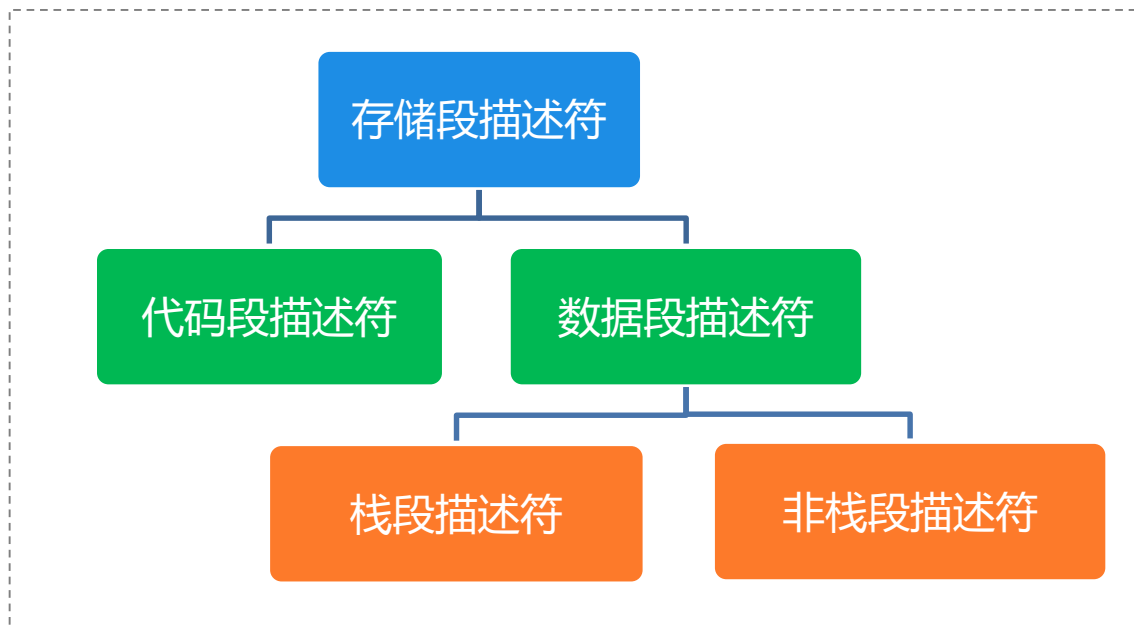


为加快查表和计算，CPU里每个选择子有对应的描述符高速缓存。

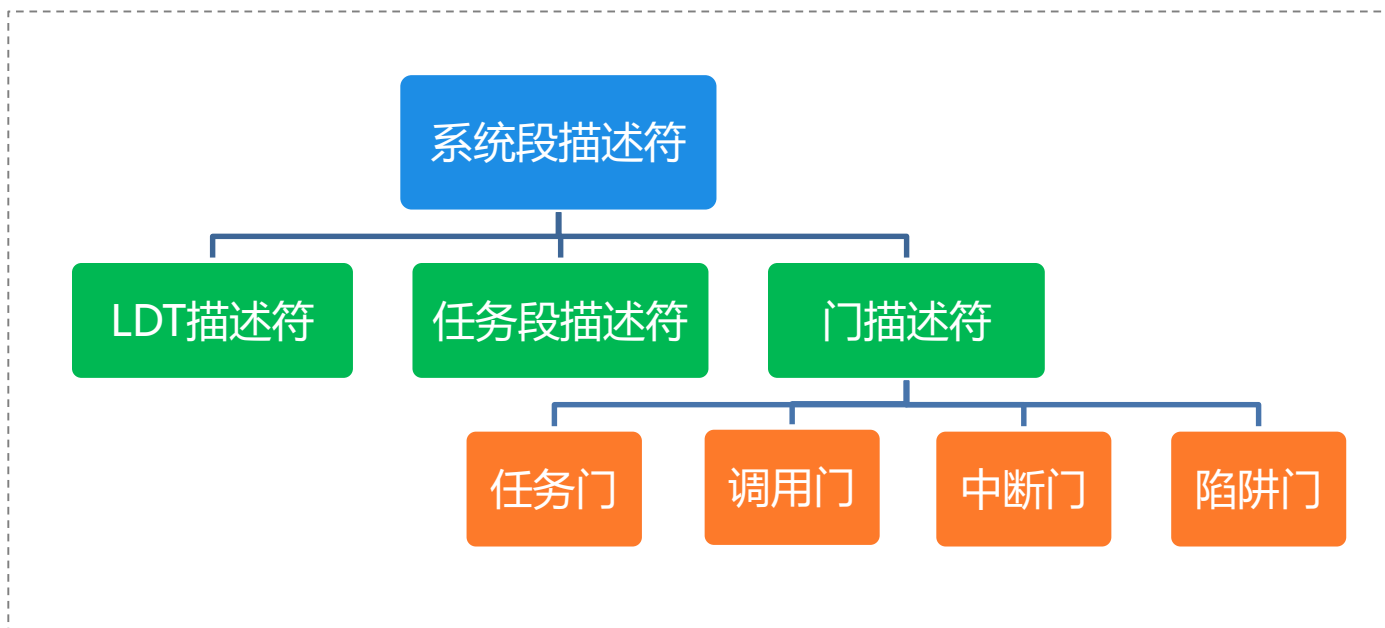


存储段描述符的分类

描述符分为存储段描述符和系统段描述符。



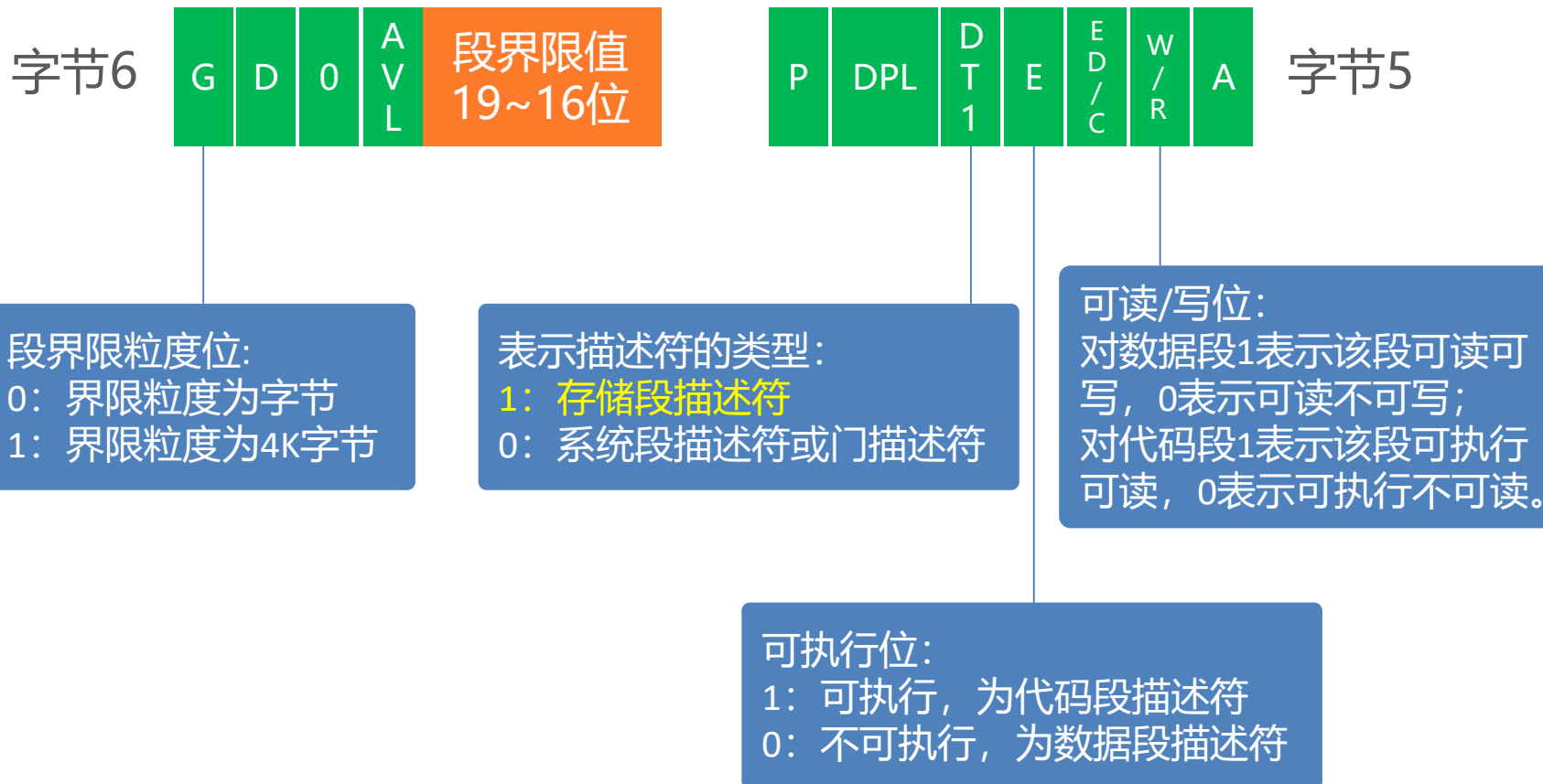
系统段描述符的分类



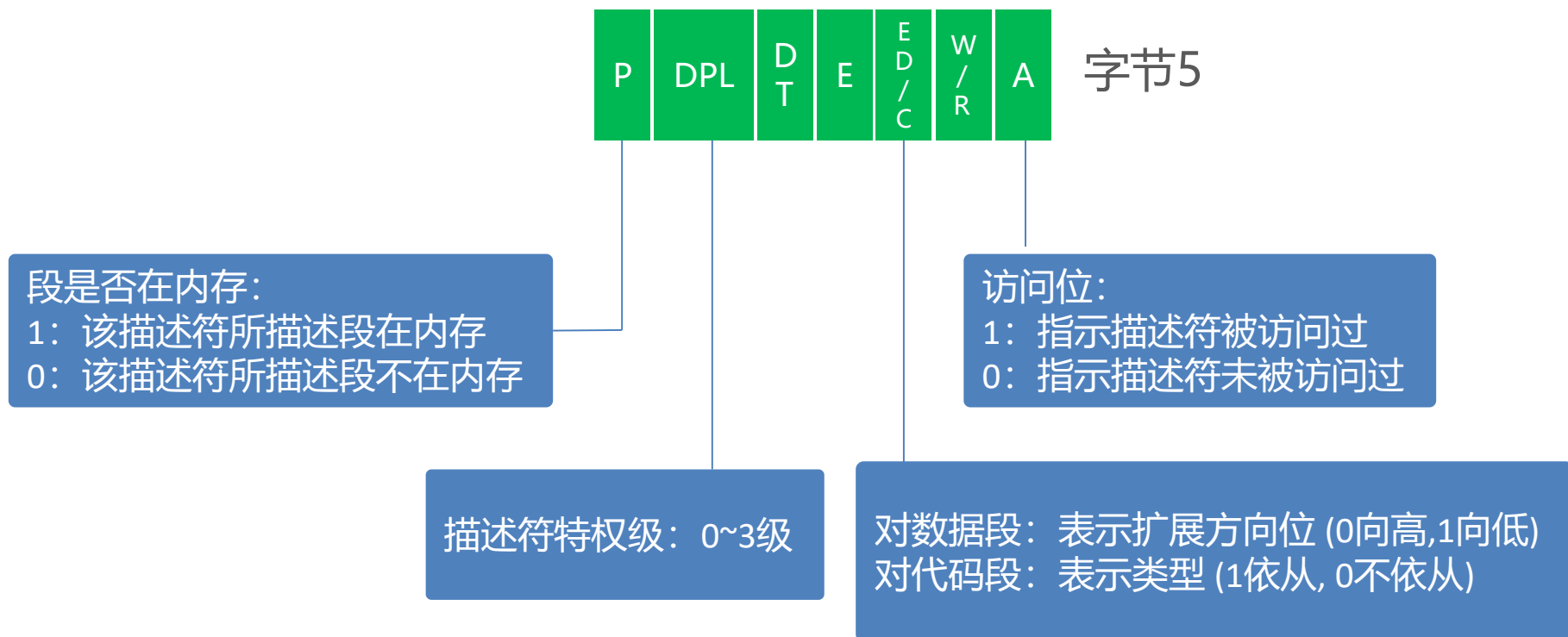
存储段（非系统）描述符格式



存储段（非系统）描述符格式



存储段（非系统）描述符格式



存储段的段界

段界限决定段的寻址范围，它由描述符中20位段限长及**G位**得到。

- **G = 0**：32位段限长的高12位为0，其低20位是段描述符中20位段限，段最大寻址范围1MB。
- **G = 1**：段限长等于描述符中规定的20位限长左移12位+ 0FFFH 。



存储段的段界

段基地址为20000000H，
段描述符中的20位限长为
0FFFFFFH。当描述符中的
G = 0时，此段的范围？

G=0:

20000000H~200FFFFFFH

1MB

段基地址为00000000H，
段描述符中的20位限长为
0FFFFFFH。当描述符中的
G = 1时，此段的范围？

G=1:

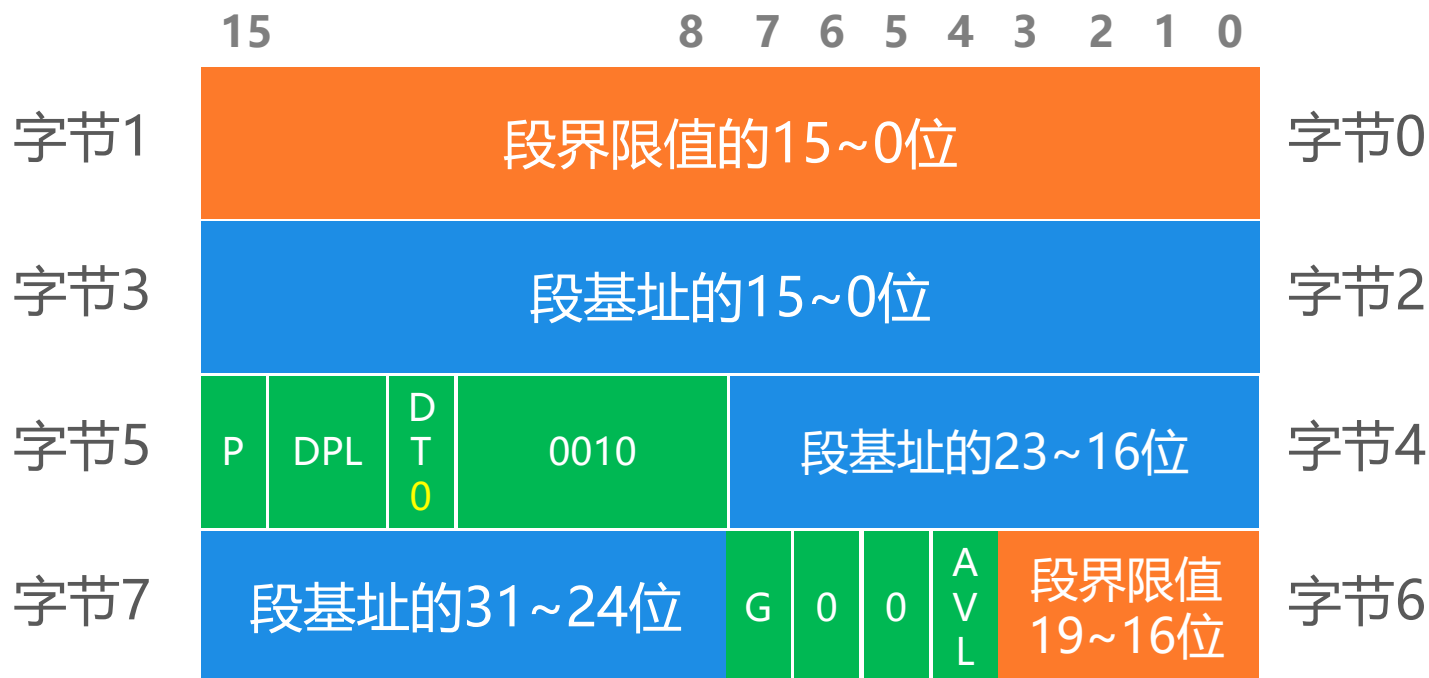
0FFFFFFH左移12位+0FFFH

段界限为: 0FFFFFFFFFH

0~ 0FFFFFFFFFH, 4GB



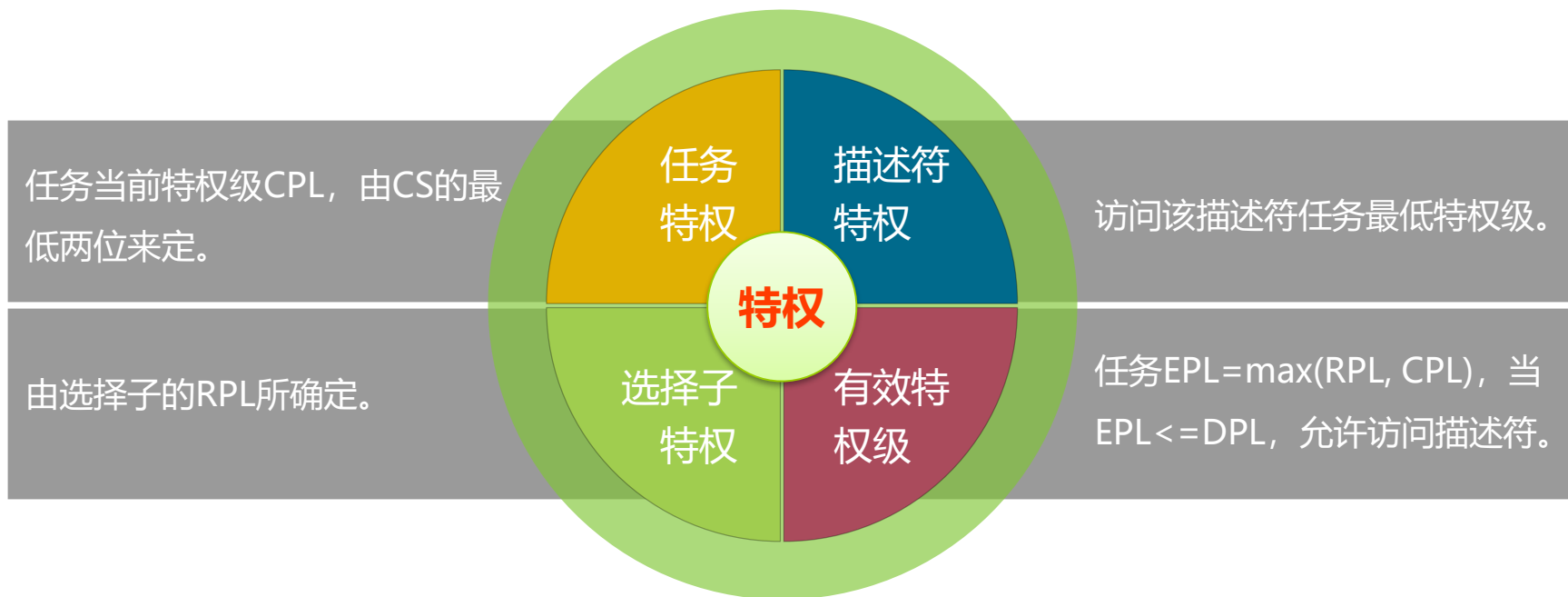
LDT描述符



LDT描述符存放LDT的基地址、界限、属性等信息。



80386的一些有关特权的概念

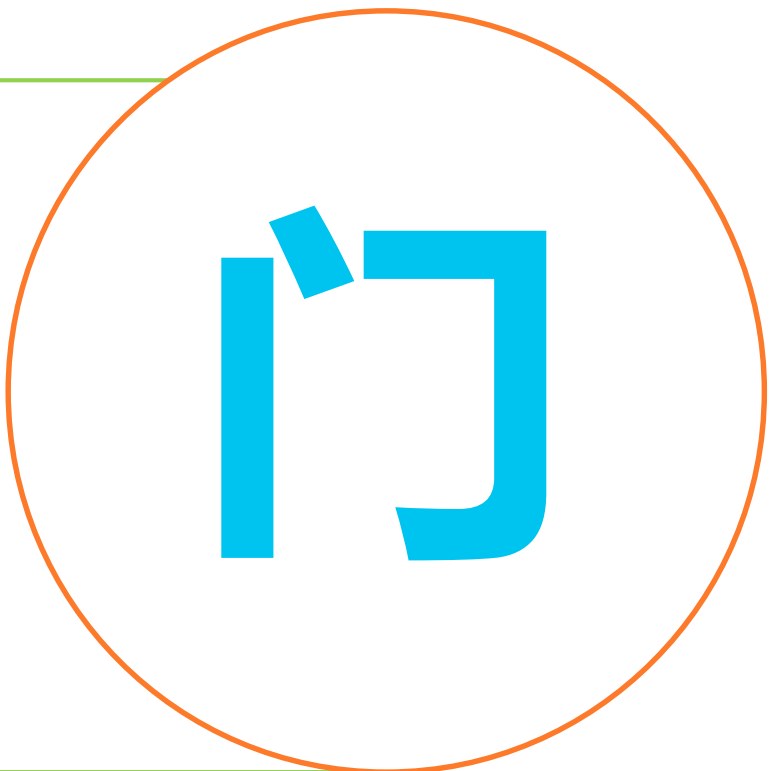


门的概念



80386允许低权限任务调用内核级代码，但是需要通过门调用。通过门调用可实现特权级的转变和任务切换。

门描述符并不描述某种内存段，而是描述控制转移的入口点，即目标代码的门。



门描述符

64位的门描述符由三部分组成：16位选择子、32位偏移地址及DPL。

调用门描述符

- 用于特权级切换。
- 存在于GDT或LDT
- 其选择子指向代码段偏移地址对应代码段中偏移量。

任务门描述符

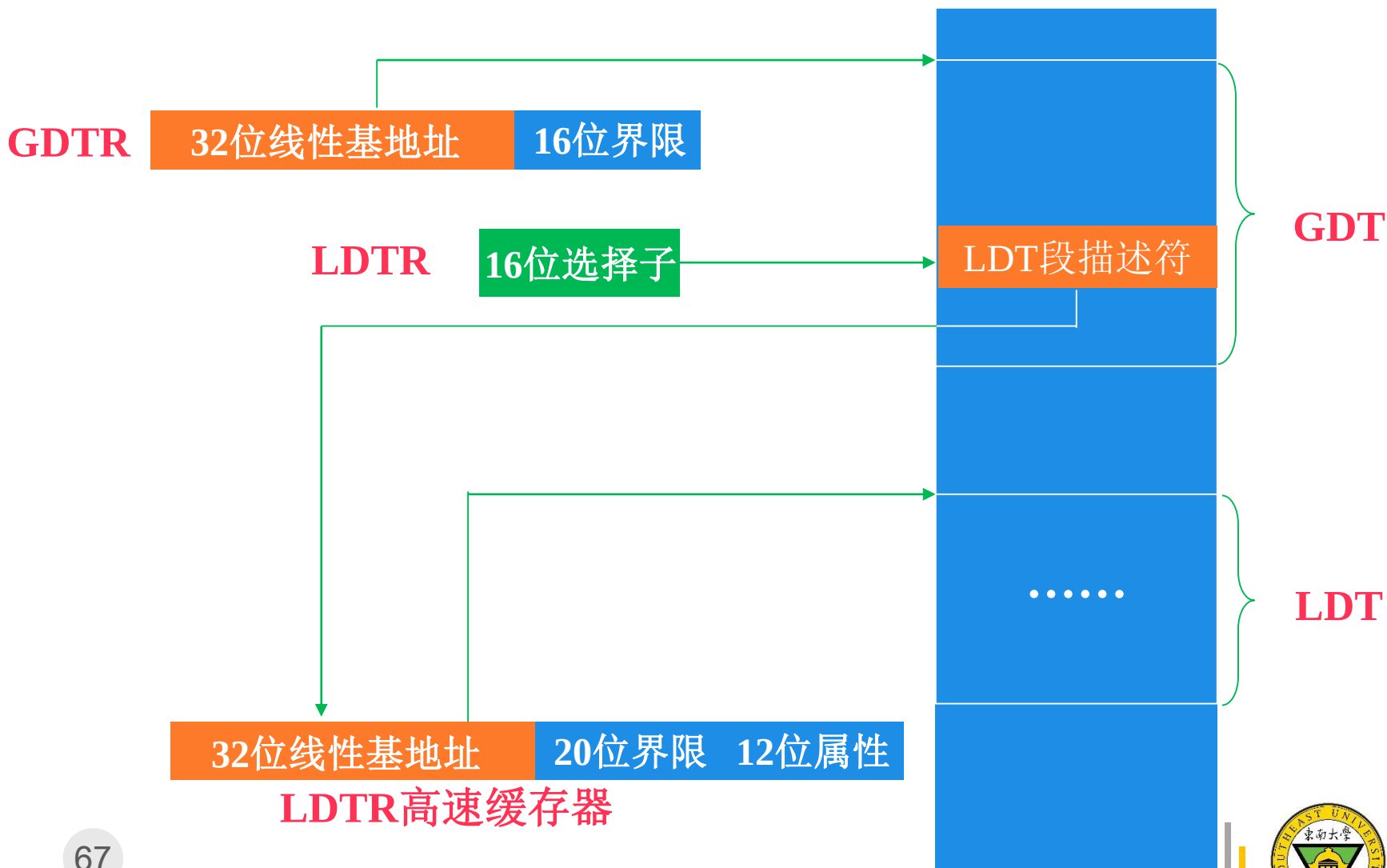
- 用于任务切换。
- 在GDT、LDT、IDT
- 选择子指向GDT中的TSS描述符。
- 偏移地址无意义。

中断门和陷阱门描述符

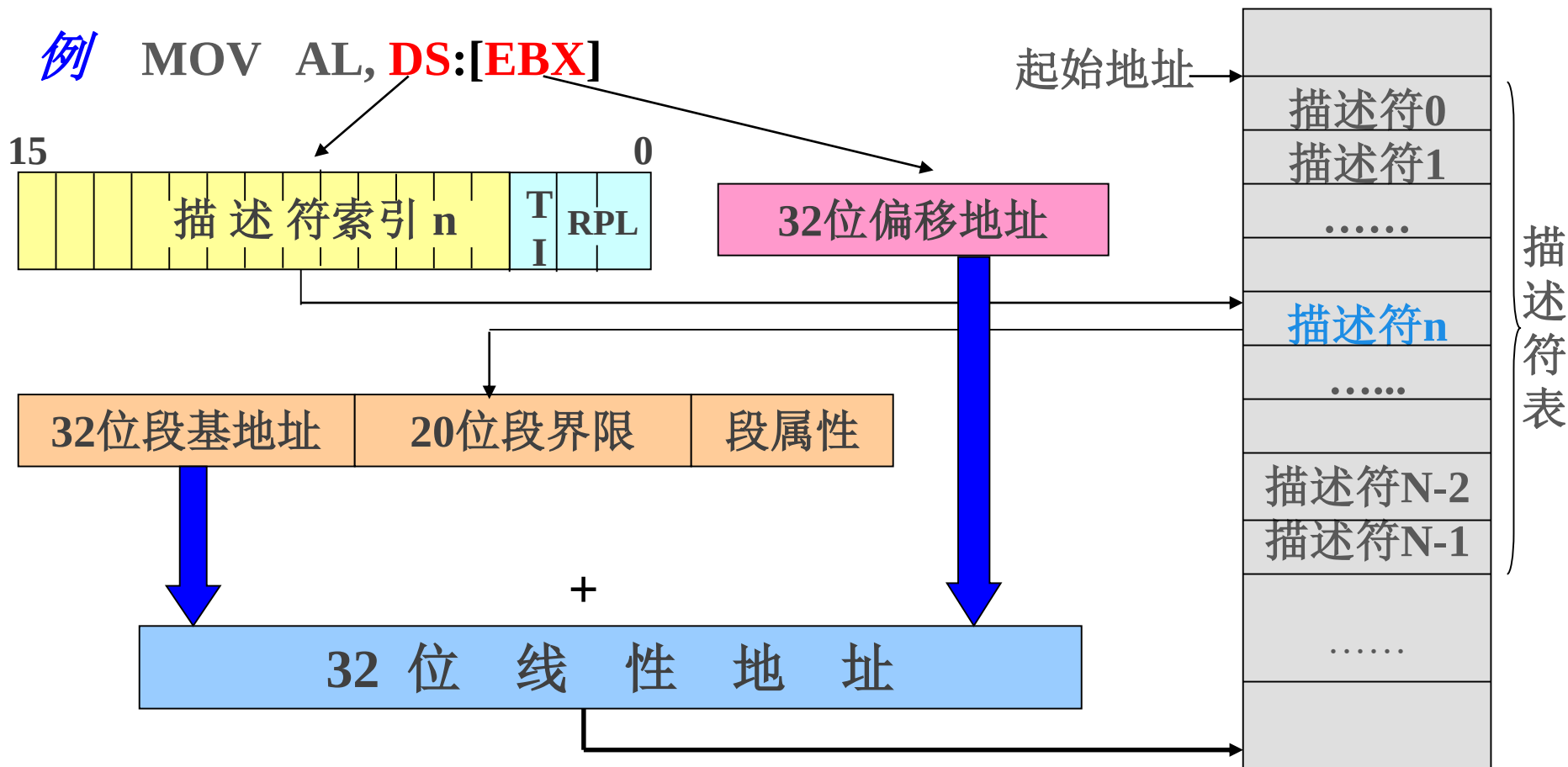
- 中断处理程序入口。
- 存在于IDT。
- 选择子指向代码段选择符、偏移地址指向代码段中。



LDTR、LDT与GDT、GDTR的关系



逻辑地址转换为线性地址



将段选择子指向的段描述符中**32位基地址**与**32位偏移地址**相加即得线性地址。

段选择器

0000 0000 0110 0100

CPU 给出的逻辑地址

DS 段选择子 0100H

①

偏移量

000CH

访问权 界限 基地址

D1 2000H 00042000H

段描述符寄存器

② 0060H

LDTR

00FFH

00100000H

00042100H

+

④

③

+

段顶

2000H

段基址

00FFH

描述符表
LDT

存储器

分段管理的小结

1

LDT和GDT实现进程的虚拟地址的隔离，每个进程有不同的LDT，LDT规定了代码段，数据段，栈段。

2

程序员只需要知道自己有4GB的线性空间使用即可。



80386存储器管理

◆ 分段

◆ 分页

◆ 举例

线性地址到物理地址

1

80386按页使用内存，每个页4KB，索引该页的地址范围是： $0 \sim 2^{12}-1$ 。4GB的线性地址空间，可以分为 2^{20} 个页。

2

页的基地址具有：“XXXXX000H”的形式，因此其高20位被称为页码。页码左移12位可得页物理基地址。

3

页为4K字节且页边界是4K的倍数，所以把32位线性地址转换成32位物理地址的过程中，低12位地址保持不变。



线性地址空间的XXXXXXH页映射到物理地址空间YYYYYYH页，那么线性地址XXXXXXxxxH被转换为YYYYYYxxxH。



线性地址如何映射为物理地址

线性地址到物理地址的转换要解决的是线性地址空间的页到物理地址空间的页的映射，也就是线性地址高20位到物理地址高20位的转换。



映射方法

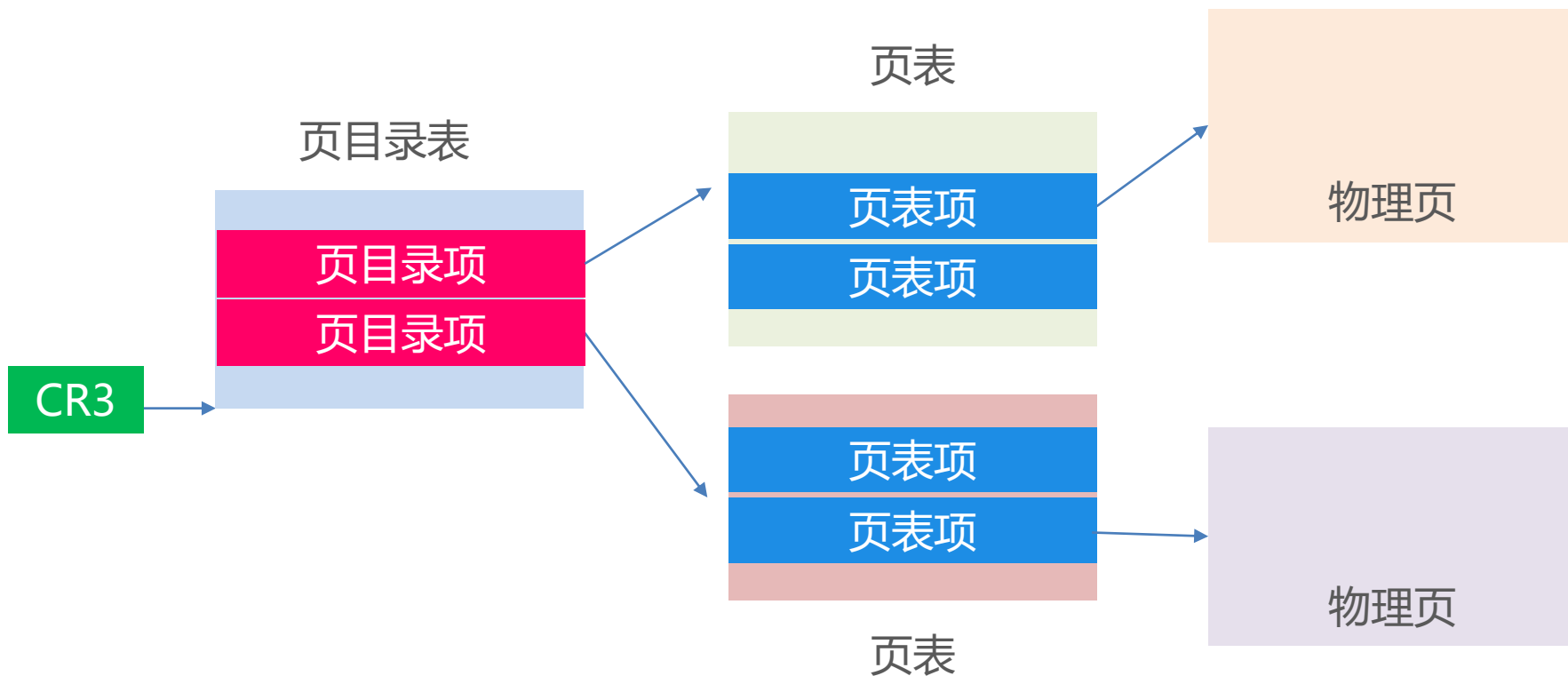


采用表格来建立线性地址页到物理页的映射， 2^{20} 个页， 2^{20} 个表项，如果每个4字节，占用4MB。开销大！

多级页映射表！

问题

线性地址到物理地址：两级页表



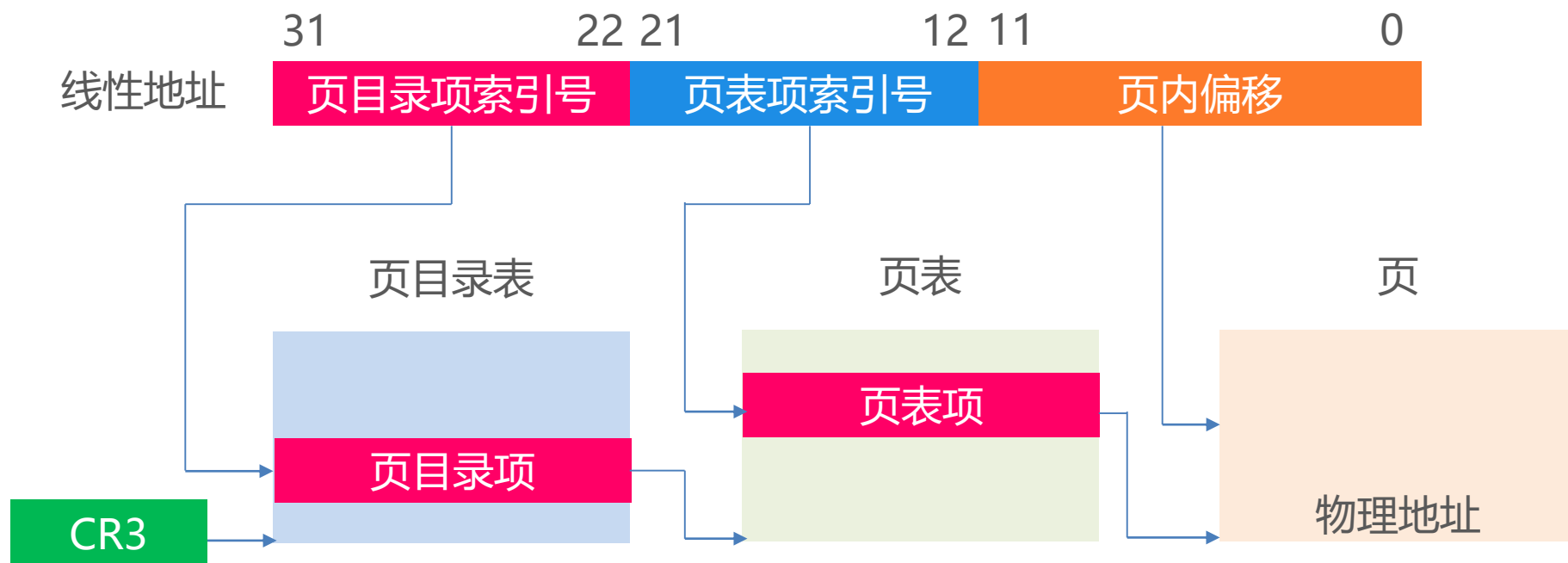
每表格4KB，CR3放页目录表基地址，页目录项放页表基地址，页表项放页基地址。



系统为每个进程分配一个页目录表，只有被使用的页表才被分配到内存。



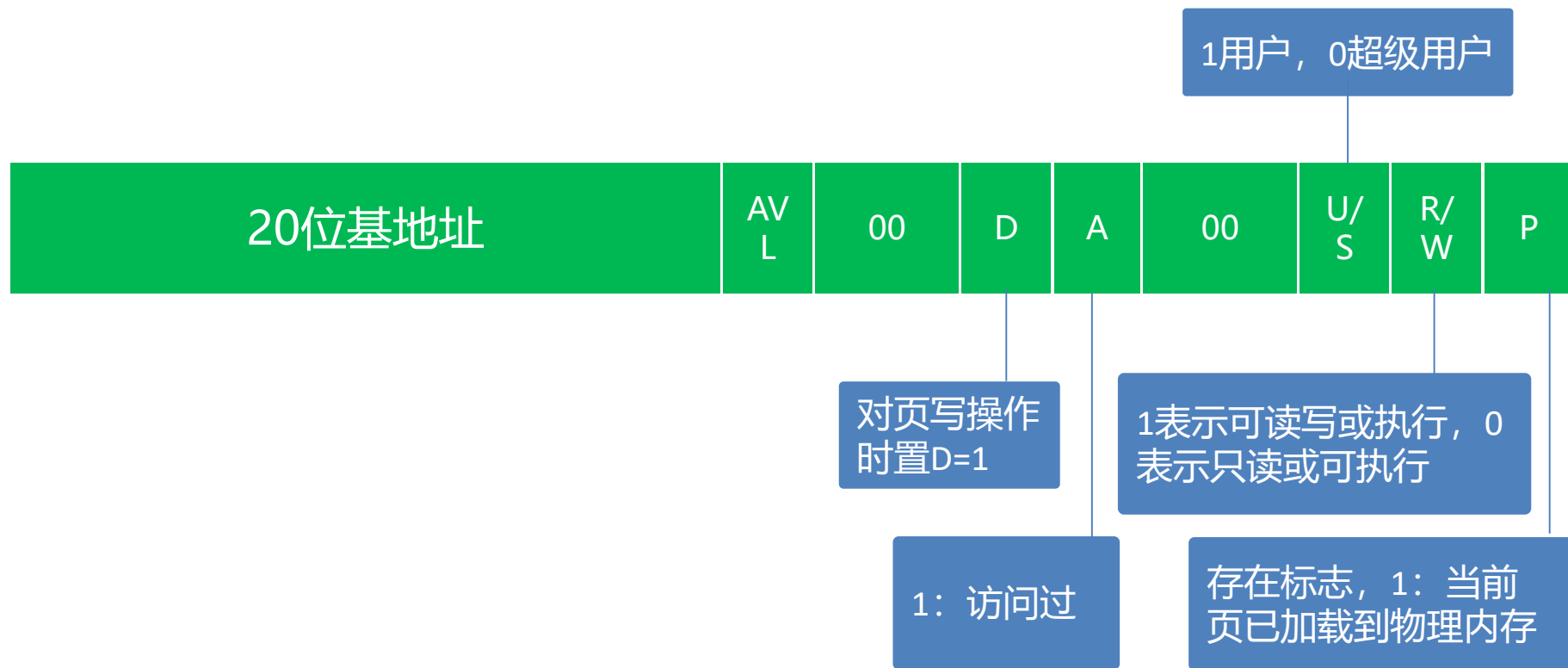
从线性地址转换成物理地址实例



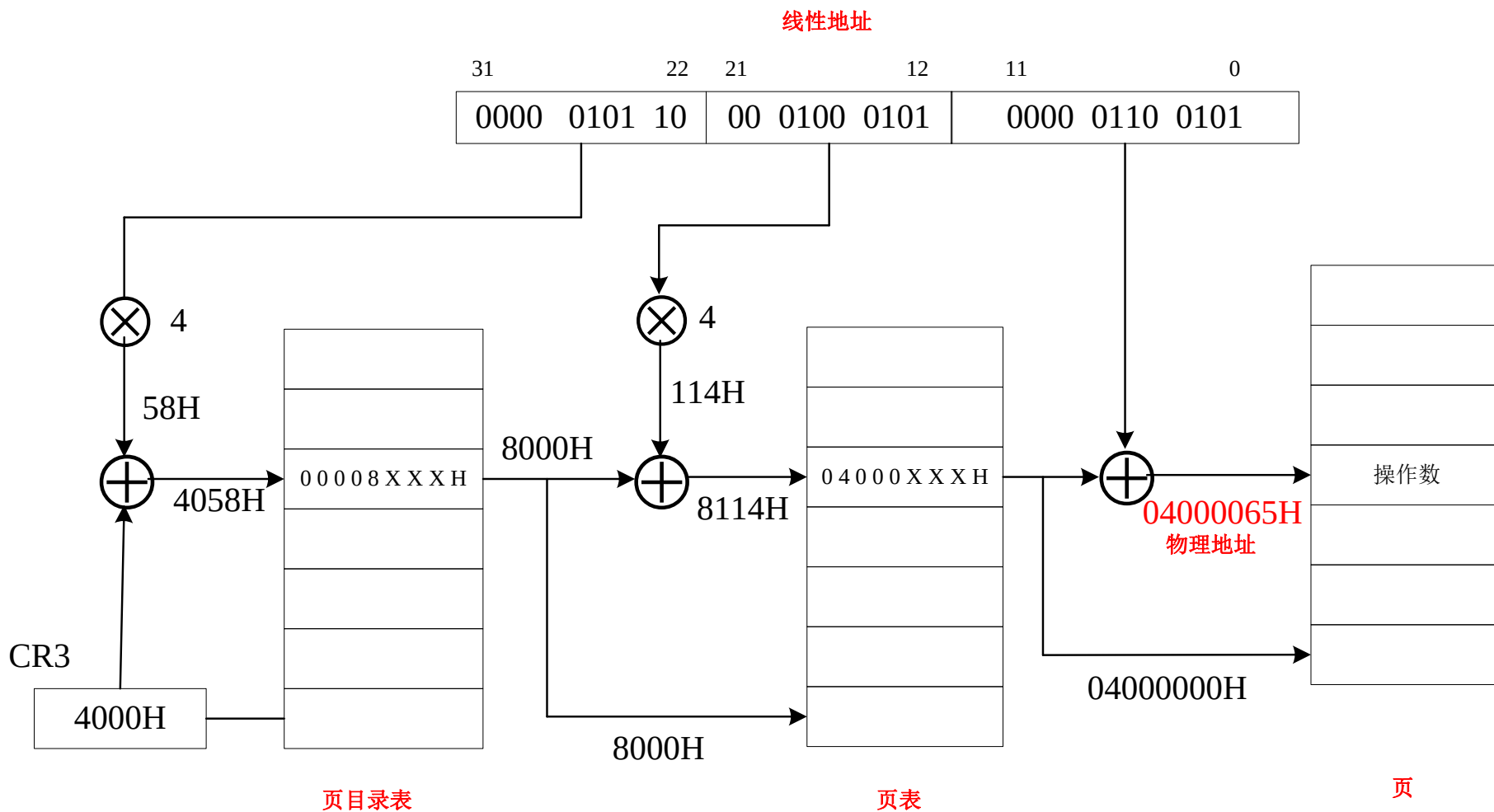
- 页目录项索引号：线性地址第31 ~ 22共10位，页目录中页目录项的索引。
- 页表项索引号：线性地址第21 ~ 12共10位，页表中页表项的索引。
- 页内偏移量：线性地址第11 ~ 0共12位，寻址页内地址。



页目录项和页表项



从线性地址转换成物理地址实例



分页管理的小结

1

每个进程都有自己的页目录表，即虚拟地址到物理地址的映射函数。

2

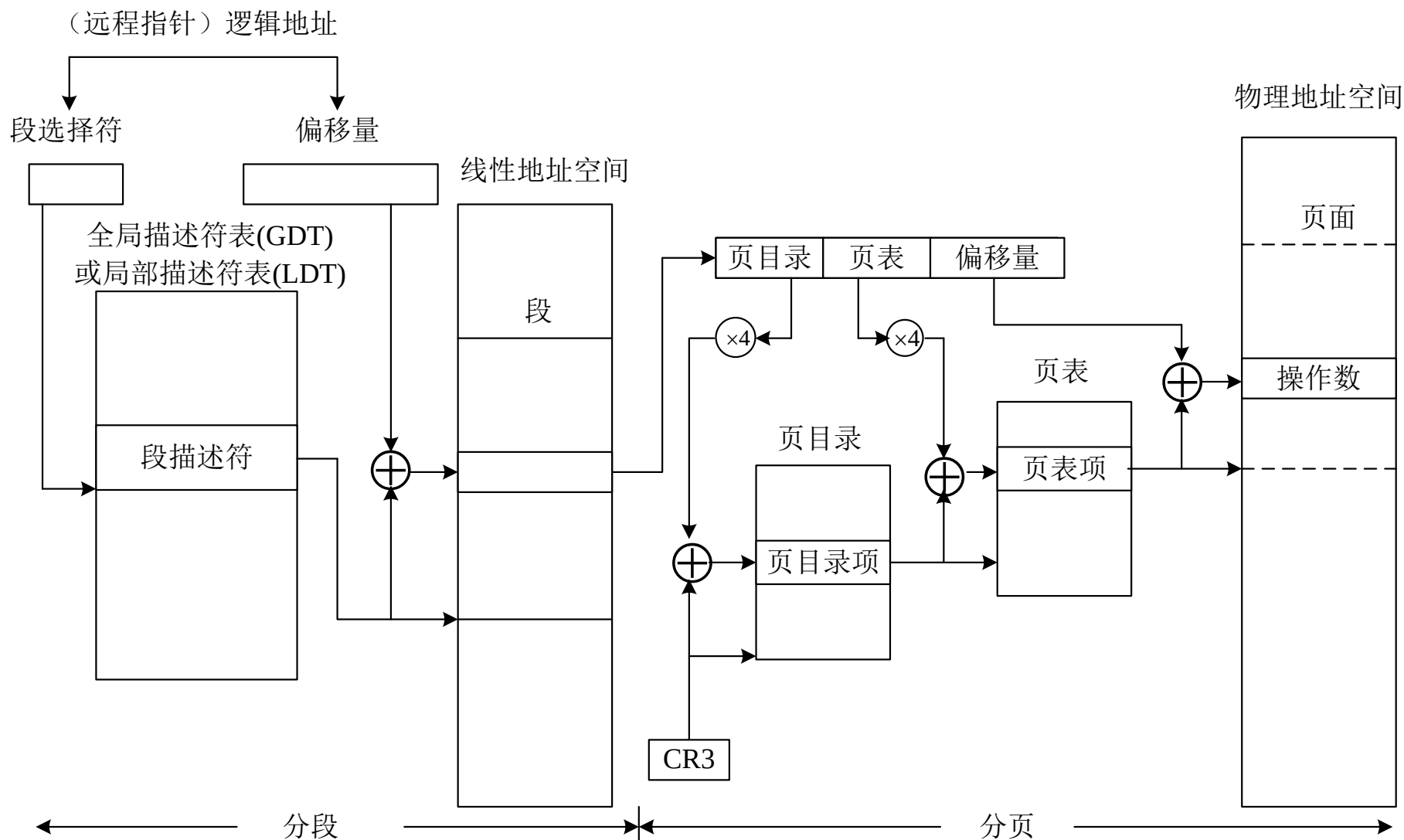
同样的线性地址，会被映射到不同的物理地址。因此，二级页表实现了物理地址的隔离。

3

程序员不必关心实际物理存储区是否连续，可将不连续的物理内存块连在一起，在线性地址上视为连续，可更有效使用内存碎片。



从逻辑地址到物理地址



在编写应用程序时，程序员不用过多地考虑系统底层是如何实现这种机制的。在进行应用程序开发时，程序员只需了解以下内容。

1

虚拟地址空间中的数据是分页管理的。应用程序不用考虑系统中其他应用程序的内存使用情况，如占用多少或占用了哪些内存等。

2

虚拟地址并不是物理地址空间中的地址，不是数据在内存中真实存在的地址，操作系统会将进程的虚拟地址映射到物理内存地址。



3

进程也不用考虑真实的物理内存有多大，只需要了解可以使用4GB（实际Linux 3GB，Win 2~3GB）内存，OS负责转换。

4

如果系统中没有足够的物理内存供使用，那么OS会将当前没有使用的内存分页“调度”到硬盘上保存起来。

5

页面调度不会造成内存中的数据在虚拟地址空间中地址的改变，所以进程无需知道分页如何调度的，无需知道数据是在内存中还是在硬盘上，只需知道其虚拟地址即可。



80386存储器管理

◆ 分段

◆ 分页

◆ 举例

80386工作在保护方式下, GDTR=0020,0000,7FFFH,
CS=4002H, 代码段描述符和数据段描述符如下:

代码段描述符

7	00	03
5	DA	60
3	00	00
1	FF	FF

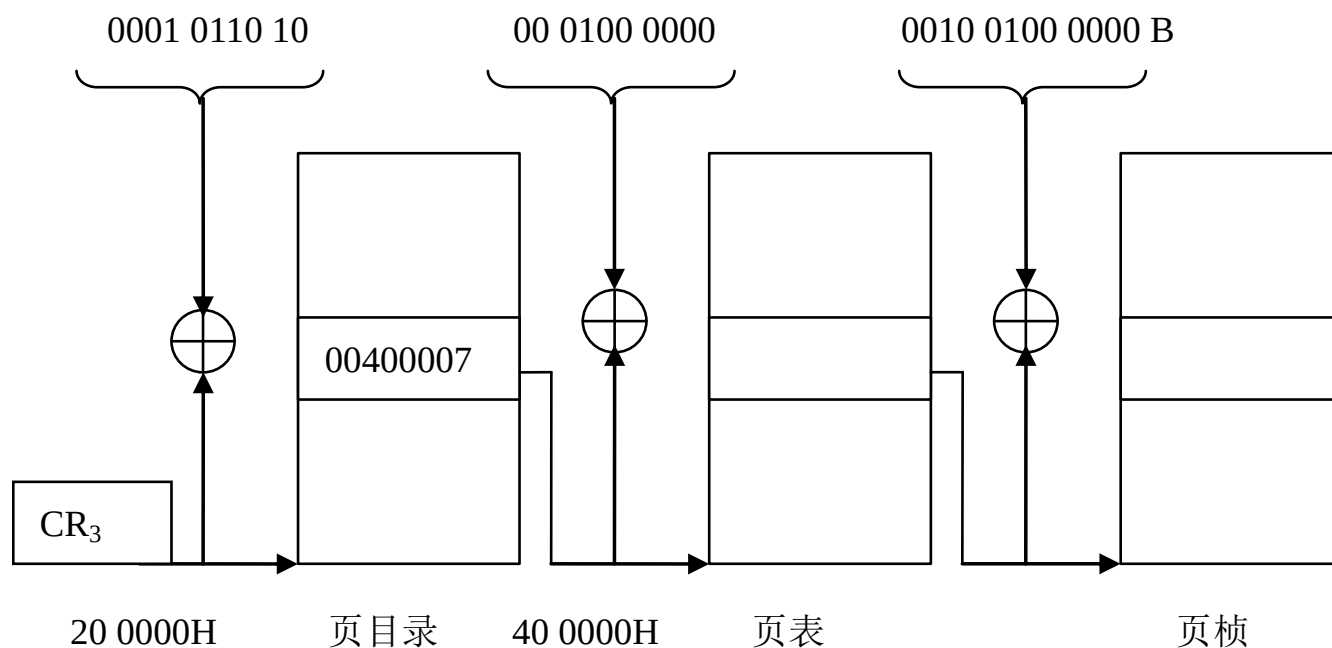
数据段描述符

00	01
F2	70
00	00
0F	FF

1. GDT的起始地址为_____, 结束地址为_____
2. 代码段描述符的起始地址为_____, 结束地址为_____
3. 代码段的起始地址为_____, 结束地址为_____
4. 数据段的起始地址为_____, 结束地址为_____
5. 代码段: 是否可读____, 在内存中否____, 访问过否____
6. 数据段: 是否可写____, 在内存中否____, 访问过否____



已知在80386系统中，页目录起始地址为20 0000H，页表起始地址为40 0000H，页目录表内容如下图所示，已将线性地址1684 0240转换为物理地址7000 0240H。



CR3=_____;

页表项偏移地址=_____;

页帧起始地址=_____;

存放整个页目录表的内存地址范围: _____~_____;

如果页表项所描述的页帧用户方式是只读, 则该页表项的内_____;



剧终

预祝大家取得好成绩!

授课
教师

王东明

移动通信国家重点实验室



Tel: 13915982595



wangdm@seu.edu.cn



无线谷1-205

